OBJECT ORINENTED PROGRAMMING WITH JAVA ASSIGNMENT

YEAR…2026……

Name: Priyanshu Kumar Tandon

Course : Bsc.Computerscience(Hons)

Topic:Assignment

Rollno: 24070154

Submitted to:

Prof. Ratnesh Shrivastava
Department of CSIT
GGV Bilaspur

1.Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

Solution:

```
import java.io.*;
import java.util.*;

class Farmer {
    String name, fruit, city;
    int quantity;

    Farmer(String name, String fruit, String city, int
quantity) {
        this.name = name;
        this.fruit = fruit;
        this.city = city;
        this.quantity = quantity;
    }
}

public class FarmerData {

    // Method to display data in tabular format
    static void display(List<Farmer> list) {
        System.out.printf("\n%-15s %-15s %-15s %-10s\n",
"Farmer Name", "Fruit", "City", "Quantity");
        System.out.println("-----------------------------------
-----------------------");
        for (Farmer f : list) {
            System.out.printf("%-15s %-15s %-15s %-10d\n",
f.name, f.fruit, f.city, f.quantity);
        }
    }
```

```java
    // Method to save data to file
    static void saveToFile(String filename, List<Farmer> list,
String title) {
        try (BufferedWriter bw = new BufferedWriter(new
FileWriter(filename, true))) {
            bw.write("\n" + title + "\n");
            bw.write(String.format("%-15s %-15s %-15s %-10s\n",
"Farmer Name", "Fruit", "City", "Quantity"));
            bw.write("----------------------------------------
-----------------\n");

            for (Farmer f : list) {
                bw.write(String.format("%-15s %-15s %-15s %-
10d\n", f.name, f.fruit, f.city, f.quantity));
            }
        } catch (IOException e) {
            System.out.println("Error writing to file.");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Farmer> farmers = new ArrayList<>();

        // Input
        System.out.println("Enter details for 10 farmers:");
        for (int i = 0; i < 10; i++) {
            System.out.println("\nFarmer " + (i + 1) + ":");

            System.out.print("Enter farmer name: ");
            String name = sc.nextLine();

            System.out.print("Enter fruit: ");
            String fruit = sc.nextLine();

            System.out.print("Enter city: ");
            String city = sc.nextLine();
```

```
            System.out.print("Enter quantity: ");
            int quantity = sc.nextInt();
            sc.nextLine(); // consume newline

            farmers.add(new Farmer(name, fruit, city,
quantity));
        }

        // Before sorting
        System.out.println("\nData Before Sorting:");
        display(farmers);
        saveToFile("farmers.txt", farmers, "Data Before
Sorting");

        // Sorting by quantity
        Collections.sort(farmers, Comparator.comparingInt(f ->
f.quantity));

        // After sorting
        System.out.println("\nData After Sorting:");
        display(farmers);
        saveToFile("farmers.txt", farmers, "Data After
Sorting");

        // Search
        System.out.print("\nEnter farmer name to search: ");
        String searchName = sc.nextLine();

        boolean found = false;
        for (Farmer f : farmers) {
            if (f.name.equalsIgnoreCase(searchName)) {
                System.out.println("\nFarmer Found:");
                System.out.printf("%-15s %-15s %-15s %-10d\n",
f.name, f.fruit, f.city, f.quantity);
                found = true;
                break;
            }
        }
```

```java
        if (!found) {
            System.out.println("Farmer not found.");
        }

        sc.close();
    }
}
```

Output:

```
Enter details of Farmer 1
Name: Ram
Fruit: Apple
City: kotgarh
Quantity: 100
Enter details of Farmer 2
Name: Mohan
Fruit: Mango
City: kurud
Quantity: 120
Enter details of Farmer 3
Name: Ankit
Fruit: Grapes
City: Nashik
Quantity: 200
Enter details of Farmer 4
Name: Ramesh
Fruit: Banana
City: Jalgaon
Quantity: 200
Enter details of Farmer 5
Name: Praveen
Fruit: Orange
City: Nagpur
Quantity: 300
Enter details of Farmer 6
Name: Anurag
Fruit: Papaya
City: Vadodra
Quantity: 300
Enter details of Farmer 7
Name: Gaurav
Fruit: Watermelon
City: Anantpur
Quantity: 500
Enter details of Farmer 8
Name: Ratnesh
Fruit: Strawberry
City: Mahabaleshwar
Quantity: 600
Enter details of Farmer 9
Name: Aman
Fruit: Guava
City: Allahabad
Quantity: 400
Enter details of Farmer 10
Name: Aditya
Fruit: Pomegranate
City: Solapur
Quantity: 300
Enter farmer name to search: Ramesh
Farmer Found!
Name: Ramesh
Fruit: Banana
City: Jalgaon
Quantity: 200
```

2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

Solution:

```java
import java.util.*;

class Fruit {
    String name;
    int quantity;
    String quality;

    Fruit(String name, int quantity, String quality) {
        this.name = name;

        // Apply condition: if quantity > 1000, store half
        if (quantity > 1000) {
            this.quantity = quantity / 2;
        } else {
            this.quantity = quantity;
        }

        this.quality = quality;
    }
}

public class FruitData {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Fixed fruits
        String[] fruitNames = {"Mango", "Apple", "Papaya"};
        List<Fruit> fruits = new ArrayList<>();
```

```java
        // Input
        for (String fname : fruitNames) {
            System.out.println("\nEnter details for " + fname +
":");

            System.out.print("Enter quantity: ");
            int qty = sc.nextInt();
            sc.nextLine(); // consume newline

            System.out.print("Enter quality (good/bad/average):
");
            String quality = sc.nextLine();

            fruits.add(new Fruit(fname, qty, quality));
        }

        // Display stored data
        System.out.println("\nStored Data:");
        System.out.printf("%-10s %-10s %-10s\n", "Fruit",
"Quantity", "Quality");
        System.out.println("--------------------------------");
        for (Fruit f : fruits) {
            System.out.printf("%-10s %-10d %-10s\n", f.name,
f.quantity, f.quality);
        }

        // Search operation
        System.out.print("\nEnter fruit name to search: ");
        String searchFruit = sc.nextLine();

        System.out.print("Enter quality to search: ");
        String searchQuality = sc.nextLine();

        boolean found = false;

        for (Fruit f : fruits) {
            if (f.name.equalsIgnoreCase(searchFruit) &&
                f.quality.equalsIgnoreCase(searchQuality)) {
```

```
                System.out.println("\nFruit Found:");
                System.out.printf("%-10s %-10d %-10s\n",
f.name, f.quantity, f.quality);
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("No matching fruit found.");
        }

        sc.close();
    }
}
```

Output:

```
Enter details for Mango:
Enter quantity: 800
Enter quality (good/bad/average): good

Enter details for Apple:
Enter quantity: 500
Enter quality (good/bad/average): bad

Enter details for Papaya:
Enter quantity: 900
Enter quality (good/bad/average): average

Stored Data:
Fruit      Quantity   Quality
--------------------------------
Mango      800        good
Apple      500        bad
Papaya     900        average

Enter fruit name to search: Mango
Enter quality to search: good

Fruit Found:
Mango      800        good
```

3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.

The available orchards in a tree-view. Search for a given fruit in the available orchards after taking input from the user & print the tree.

Solution :

```
import java.util.*;

// Tree Node class
class Node {
    String orchard;
    String fruit;
    String quality;
    List<Node> children;

    Node(String orchard, String fruit, String quality) {
        this.orchard = orchard;
        this.fruit = fruit;
        this.quality = quality;
        this.children = new ArrayList<>();
    }
}

public class OrchardTree {

    // Method to print tree
    static void printTree(Node root, String searchFruit) {
        System.out.println("\nTree Structure:\n");

        System.out.println(root.orchard); // Root

        for (Node child : root.children) {
            if (child.fruit.equalsIgnoreCase(searchFruit)) {
                System.out.println(" ├── " + child.orchard +
                    " -> " + child.fruit + " (" + child.quality
+ ")  [FOUND]");
```

```
            } else {
                System.out.println(" ├── " + child.orchard +
                    " -> " + child.fruit + " (" + child.quality
+ ")");
            }
        }
    }

    // Search function
    static boolean search(Node root, String fruit) {
        for (Node child : root.children) {
            if (child.fruit.equalsIgnoreCase(fruit)) {
                return true;
            }
        }
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Create root
        Node root = new Node("Orchards", "", "");

        // Create orchard nodes
        root.children.add(new Node("A", "Apple", "good"));
        root.children.add(new Node("B", "Litchi", "good"));
        root.children.add(new Node("C", "Mango", "good"));
        root.children.add(new Node("D", "Apple", "bad"));

        // User input
        System.out.print("Enter fruit to search: ");
        String searchFruit = sc.nextLine();

        // Search
        boolean found = search(root, searchFruit);

        if (found) {
```

```java
            System.out.println("\nFruit found in orchard(s).");
        } else {
            System.out.println("\nFruit not found.");
        }

        // Print tree
        printTree(root, searchFruit);

        sc.close();
    }
}
```

Output:

```
Enter fruit to search: Apple


Fruit found in orchard(s).


Tree Structure:


Orchards
  ├── A -> Apple (good)  [FOUND]
  ├── B -> Litchi (good)
  ├── C -> Mango (good)
  ├── D -> Apple (bad)   [FOUND]
```

4.Take a map provided as a graph & display all the subtrees of connected points.

Solution:

```
import java.util.*;

public class GraphSubtrees {

    // DFS function
    static void dfs(int node, boolean[] visited,
List<List<Integer>> graph, List<Integer> component) {
        visited[node] = true;
        component.add(node);

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                dfs(neighbor, visited, graph, component);
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int v = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        // Adjacency list
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < v; i++) {
            graph.add(new ArrayList<>());
        }

        // Input edges
        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
```

```java
            int u = sc.nextInt();
            int w = sc.nextInt();

            graph.get(u).add(w);
            graph.get(w).add(u); // undirected graph
        }

        boolean[] visited = new boolean[v];

        System.out.println("\nConnected Components
(Subtrees):");

        // Find all connected components
        for (int i = 0; i < v; i++) {
            if (!visited[i]) {
                List<Integer> component = new ArrayList<>();
                dfs(i, visited, graph, component);

                // Print component
                System.out.println(component);
            }
        }

        sc.close();
    }
}
```

Output:

```
Vertices = 6
Edges = 4
0 1
1 2
3 4
4 5
Connected Components (Subtrees):
[0, 1, 2]
[3, 4, 5]
```

5.Enter entry point & goal point & produce the matching subtree of the graph.

Solution:

```
import java.util.*;

public class GraphPath {

    // Function to find path using DFS
    static boolean dfs(int current, int goal, boolean[]
visited,
                       List<List<Integer>> graph, List<Integer>
path) {

        visited[current] = true;
        path.add(current);

        // If goal reached
        if (current == goal) {
            return true;
        }

        // Visit neighbors
        for (int neighbor : graph.get(current)) {
            if (!visited[neighbor]) {
                if (dfs(neighbor, goal, visited, graph, path))
{
                    return true;
                }
            }
        }

        // Backtrack if not found
        path.remove(path.size() - 1);
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```java
        System.out.print("Enter number of vertices: ");
        int v = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        List<List<Integer>> graph = new ArrayList<>();

        for (int i = 0; i < v; i++) {
            graph.add(new ArrayList<>());
        }

        // Input edges
        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int w = sc.nextInt();

            graph.get(u).add(w);
            graph.get(w).add(u); // undirected graph
        }

        // Input entry & goal
        System.out.print("Enter entry point: ");
        int start = sc.nextInt();

        System.out.print("Enter goal point: ");
        int goal = sc.nextInt();

        boolean[] visited = new boolean[v];
        List<Integer> path = new ArrayList<>();

        // Find path
        if (dfs(start, goal, visited, graph, path)) {
            System.out.println("\nMatching Subtree (Path): " +
path);
        } else {
```

```
            System.out.println("\nNo path exists between given points.");
        }

        sc.close();
    }
}
```

Output:

```
Enter number of vertices: 5
Enter number of edges: 5
Enter edges (u v):
0 1
1 2
2 3
3 4
1 4
Enter entry point: 0
Enter goal point: 4

Matching Subtree (Path): [0, 1, 2, 3, 4]
```

6.Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

Solution:

```
import java.util.*;

class DijkstraExample {

    static class Node implements Comparable<Node> {
        String vertex;
        int distance;

        Node(String vertex, int distance) {
            this.vertex = vertex;
            this.distance = distance;
        }

        public int compareTo(Node other) {
            return this.distance - other.distance;
        }
    }

    public static void dijkstra(Map<String, List<Node>> graph,
String start, String end) {
        PriorityQueue<Node> pq = new PriorityQueue<>();
        Map<String, Integer> distances = new HashMap<>();
        Map<String, String> previous = new HashMap<>();

        // Initialize distances
        for (String vertex : graph.keySet()) {
            distances.put(vertex, Integer.MAX_VALUE);
        }

        distances.put(start, 0);
        pq.add(new Node(start, 0));

        while (!pq.isEmpty()) {
            Node current = pq.poll();
```

```
            if (current.vertex.equals(end)) {
                break;
            }

            for (Node neighbor : graph.get(current.vertex)) {
                int newDist = distances.get(current.vertex) +
neighbor.distance;

                if (newDist < distances.get(neighbor.vertex)) {
                    distances.put(neighbor.vertex, newDist);
                    previous.put(neighbor.vertex,
current.vertex);
                    pq.add(new Node(neighbor.vertex, newDist));
                }
            }
        }

        // Reconstruct path
        List<String> path = new ArrayList<>();
        String step = end;

        while (step != null) {
            path.add(step);
            step = previous.get(step);
        }

        Collections.reverse(path);

        // Output
        System.out.println("Shortest Distance: " +
distances.get(end));
        System.out.println("Shortest Path: " + String.join(" ->
", path));
    }

    public static void main(String[] args) {

        // Graph representation
```

```java
        Map<String, List<Node>> graph = new HashMap<>();

        graph.put("A", Arrays.asList(new Node("B", 1), new
Node("C", 4)));
        graph.put("B", Arrays.asList(new Node("D", 2), new
Node("C", 5)));
        graph.put("C", Arrays.asList(new Node("D", 1)));
        graph.put("D", new ArrayList<>());

        dijkstra(graph, "A", "D");
    }
}
```

Output:

```
Shortest Distance: 3

Shortest Path: A -> B -> D
```

7.Write a program to demonstrate the use of all keywords made for exception handling.

Solution:

```
class ExceptionDemo {

    // Method using "throws"
    static void checkAge(int age) throws Exception {
        if (age < 18) {
            // "throw" keyword
            throw new Exception("Age is less than 18. Not
eligible.");
        } else {
            System.out.println("Eligible to vote.");
        }
    }

    public static void main(String[] args) {

        try {
            // Code that may cause exception
            int a = 10, b = 0;
            int result = a / b;   // ArithmeticException

            System.out.println("Result: " + result);

            // Calling method that may throw exception
            checkAge(16);

        }
        catch (ArithmeticException e) {
            // Handling specific exception
            System.out.println("Arithmetic Exception caught:
Division by zero");
        }
        catch (Exception e) {
            // Handling general exception
            System.out.println("Exception caught: " +
e.getMessage());
```

```
        }
        finally {
            // Always executes
            System.out.println("Finally block executed.");
        }
    }
}
```

Output:

```
Arithmetic Exception caught: Division by zero

Finally block executed.
```

8.Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

Solution:

```
// Step 1: Custom Exception Class
class NoMoneyException extends Exception {
    public NoMoneyException(String message) {
        super(message);
    }
}
// Step 2: BankAccount class with withdraw logic
class BankAccount {
    private String owner;
    private double balance;

    public BankAccount(String owner, double balance) {
        this.owner = owner;
        this.balance = balance;
    }
      public void withdraw(double amount) throws
NoMoneyException {
        if (amount > balance) {
            throw new NoMoneyException(
                "Insufficient funds! Tried to withdraw ₹" +
amount +
                " but balance is only ₹" + balance
            );
        }
        balance -= amount;
        System.out.println("₹" + amount + " withdrawn
successfully.");
        System.out.println("Remaining balance: ₹" + balance);
    }
      public double getBalance() {
        return balance;
    }
}
     // Step 3: Main class to test
    public class BankDemo {
```

```java
    public static void main(String[] args) {

        BankAccount account = new BankAccount("Rahul", 5000.0);

        // Test 1: Valid withdrawal
        try {
            System.out.println("--- Attempt 1: Withdraw ₹2000 -
--");
            account.withdraw(2000);
        } catch (NoMoneyException e) {
            System.out.println("Error: " + e.getMessage());
        }
        // Test 2: Insufficient funds
        try {
            System.out.println("\n--- Attempt 2: Withdraw ₹8000
---");
            account.withdraw(8000);
        } catch (NoMoneyException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output:

```
--- Attempt 1: Withdraw ₹2000 ---
₹2000.0 withdrawn successfully.
Remaining balance: ₹3000.0
 --- Attempt 2: Withdraw ₹8000 ---
Error: Insufficient funds! Tried to withdraw ₹8000.0
but balance is only ₹3000.0
```

9.Write a program which enables to use a platform by a train one at a time.

Solution:

```
import java.util.concurrent.Semaphore;

// Platform class controlling access
class Platform {
    private Semaphore semaphore;

    public Platform() {
        // Only 1 permit → only one train allowed at a time
        semaphore = new Semaphore(1);
    }

    public void arrive(String trainName) {
        try {
            System.out.println(trainName + " is waiting for the
platform...");
            semaphore.acquire(); // request access

            System.out.println(trainName + " has entered the
platform.");
            Thread.sleep(2000); // simulate time spent on
platform

            System.out.println(trainName + " is leaving the
platform.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            semaphore.release(); // free the platform
        }
    }
}
// Train class as a thread
class Train extends Thread {
    private Platform platform;
    private String name;
```

```java
    public Train(Platform platform, String name) {
        this.platform = platform;
        this.name = name;
    }

    public void run() {
        platform.arrive(name);
    }
}
// Main class
public class RailwaySystem {
    public static void main(String[] args) {
        Platform platform = new Platform();

        // Multiple trains trying to use the same platform
        Train t1 = new Train(platform, "Train A");
        Train t2 = new Train(platform, "Train B");
        Train t3 = new Train(platform, "Train C");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Output:

```
Train C is waiting for the platform...
Train B is waiting for the platform...
Train A is waiting for the platform...
Train C has entered the platform.
Train C is leaving the platform.
Train B has entered the platform.
Train B is leaving the platform.
Train A has entered the platform.
Train A is leaving the platform.
```

10.Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free.

Solution:

```
class Kitchen {
    private boolean orderPlaced = false;
    private String order;

    // Waiter places order
    public synchronized void placeOrder(String orderName) {
        try {
            while (orderPlaced) {
                wait(); // wait if kitchen is busy
            }
            order = orderName;
            System.out.println("Waiter placed order: " +
order);
            orderPlaced = true;

            notify(); // notify chef
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    // Chef prepares and delivers order
    public synchronized void prepareOrder() {
        try {
            while (!orderPlaced) {
                wait(); // wait if no order
            }
            System.out.println("Chef is preparing: " + order);
            Thread.sleep(2000); // simulate cooking time

            System.out.println("Chef delivered: " + order);
            orderPlaced = false;

            notify(); // notify waiter
        } catch (InterruptedException e) {
```

```
            e.printStackTrace();
        }
    }
}

// Waiter thread
class Waiter extends Thread {
    private Kitchen kitchen;

    public Waiter(Kitchen kitchen) {
        this.kitchen = kitchen;
    }

    public void run() {
        String[] orders = {"Pizza", "Burger", "Pasta"};

        for (String order : orders) {
            kitchen.placeOrder(order);
            try {
                Thread.sleep(1000); // time between orders
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

// Chef thread
class Chef extends Thread {
    private Kitchen kitchen;

    public Chef(Kitchen kitchen) {
        this.kitchen = kitchen;
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            kitchen.prepareOrder();
```

```java
        }
    }
}

// Main class
public class RestaurantSystem {
    public static void main(String[] args) {
        Kitchen kitchen = new Kitchen();

        Waiter waiter = new Waiter(kitchen);
        Chef chef = new Chef(kitchen);

        waiter.start();
        chef.start();
    }
}
```

Output:

```
Waiter placed order: Pizza
Chef is preparing: Pizza
Chef delivered: Pizza
Waiter placed order: Burger
Chef is preparing: Burger
Chef delivered: Burger
Waiter placed order: Pasta
Chef is preparing: Pasta
Chef delivered: Pasta
```